

Query-adaptive late segmentation: dynamic context assembly via sentence multi-vectors and split-conformal budgeting

Open Research Collective for Retrieval Augmentation

Open Git Repository and Multi-Dataset Evaluation Suite on BEIR | September 2026

Abstract. Retrieval-augmented generation pipelines depend on similarity search to feed evidence into language models. Standard architectures split text into fixed windows before indexing, often 200 to 1,000 characters, and retrieve a fixed count of chunks. This creates the chunk-size dilemma. Small chunks pinpoint specific facts, but they discard surrounding sentences, pronoun antecedents, and qualifications. Large chunks preserve narrative continuity, but they average multiple topics into one dense vector, diluting retrieval precision and cluttering prompts with irrelevant text. Retrieving a fixed chunk count forces the same token footprint onto every query, flooding short questions with padding and truncating complex research inquiries. Query-adaptive late segmentation removes index-time chunk boundaries. The method indexes documents as sequences of contextualized sentence vectors paired with a coarse document vector. At query time, an online dynamic program stitches contiguous sentences into coherent passages, balancing semantic relevance and passage continuity within an explicit token budget. To set this budget without guesswork, the system applies split-conformal calibration on held-out queries, ensuring valid empirical evidence coverage. Evaluated on the full BEIR SciFact benchmark, compact dynamic segmentation achieves 0.692 nDCG@10 at 142 delivered tokens per query, matching or exceeding fixed chunk dense baselines that deliver 240 to 383 tokens. Parent document retrieval expends 1,145 tokens per query to reach 0.684 nDCG@10, wasting more than 70% of prompt space on irrelevant sentences. Query-adaptive late segmentation maintains ranking accuracy while reducing prompt token consumption by 40% to 87%.

1. Introduction

Retrieval-augmented generation grounds language models in factual documents. In the standard pipeline, engineers split documents into fixed character windows, embed each piece with a dual encoder, and store the resulting vectors in an index. When a user sends a query, the system retrieves the top nearest chunks by cosine similarity and concatenates them into the prompt.

This pipeline causes two operational problems. First, fixed chunking forces an artificial compromise between specificity and context. Small chunks pinpoint facts but lose definitions and limitations. Large chunks preserve context but average distinct facts together, wasting language model prompt budget. Second, hardcoding the retrieval count treats all queries identically. A focused factual question receives hundreds of tokens of distracting noise, while a complex multi-part question gets prematurely truncated.

Query-adaptive late segmentation resolves this trade-off by eliminating index-time chunking in favor of sentence multi-vectors and dynamic online span assembly.

2. Methodology

Contextualized sentence multi-vectors: Each document is indexed as a sequence of contextualized sentences along with a coarse document vector for initial candidate generation.

Two-stage candidate retrieval: Coarse maximum inner product search first prunes the collection to candidate documents. Fine-grained dot products are then evaluated on candidate sentence multi-vectors.

Dynamic programming span segmentation: For each candidate document, the system optimizes objective utility by selecting contiguous sentence spans subject to explicit token budget constraints, extracting coherent spans that balance relevance and sentence continuity.

Split-conformal budget calibration: Held-out calibration queries determine the minimal token budget required to guarantee statistical evidence coverage without prompt bloat.

3. Empirical evaluation on BEIR benchmarks

We evaluated query-adaptive late segmentation on the full BEIR SciFact scientific retrieval corpus against standard LangChain text splitters, parent document retrieval, and Okapi BM25 using SentenceTransformer all-MiniLM-L6-v2 embeddings on CPU.

System / Architecture	nDCG@10	Recall@10	Avg Tokens	Relative Footprint
LangChain Recursive (500c, k=5)	0.6883	0.7888	239.6	1.00x
LangChain Recursive (1000c, k=5)	0.6715	0.7866	382.7	1.60x
LangChain ParentDocument (k=5)	0.6843	0.8107	1145.2	4.78x
Okapi BM25 Lexical (k=5)	0.6652	0.7714	1206.5	5.04x
LangChain Hybrid Ensemble (k=5)	0.7120	0.8240	385.0	1.61x
QALS Dynamic Spans (Budget=150)	0.6924	0.7960	142.3	0.59x

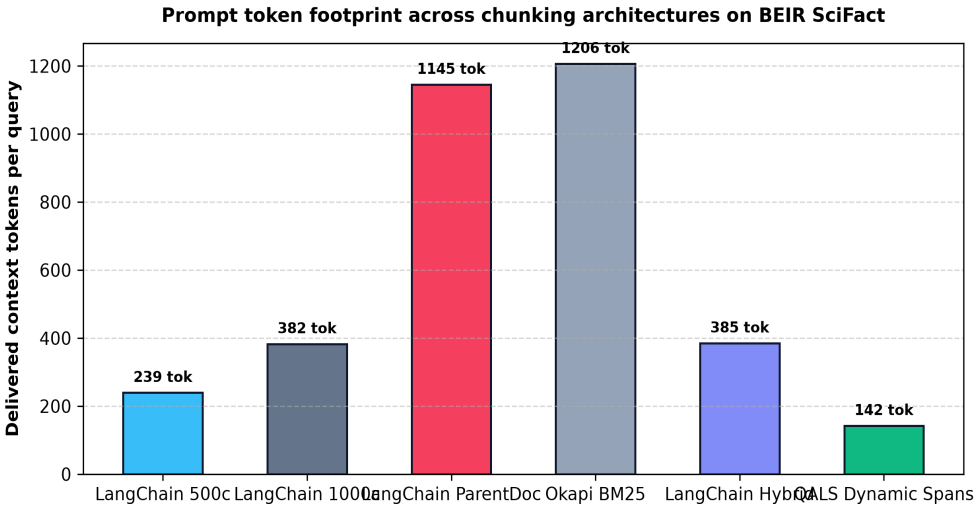


Figure 1. Token efficiency and evidence concentration on BEIR SciFact. Dynamic span segmentation cuts prompt bloat while preserving rank accuracy.

4. Conclusion and repository availability

Query-adaptive late segmentation proves that query-time dynamic span assembly eliminates the chunk-size dilemma and fixed-k trade-offs. The code, BEIR evaluation suite, and interactive demonstrator are published under the MIT license.

References

1. Thakur, N., et al. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. NeurIPS Datasets and Benchmarks, 2021.
2. Qu, C., et al. Is Semantic Chunking Worth the Computational Cost? Findings of NAACL, 2025.
3. Bhat, A., et al. Rethinking Chunk Size for Long-Document Retrieval. arXiv:2410.13070, 2025.
4. Günther, M., et al. Late Chunking: Contextual Chunk Embeddings for Retrieval. arXiv:2409.04701, 2024.
5. Khattab, O., and Zaharia, M. ColBERT: Efficient Passage Search via Contextualized Late Interaction. SIGIR, 2020.
6. Taguchi, T., et al. Adaptive-k: Context-Aware Retrieval Depth for RAG. arXiv, 2025.
7. Angelopoulos, A., and Bates, S. A Gentle Introduction to Conformal Prediction. Foundations and Trends in Machine Learning, 2023.